

Esempi svolti in pseudo-ADA

Realizzazione tipo stretto Dizionario con Chiave di tipo
Intero e Valore di tipo Carattere

~ ~ ~ ~ ~

package Type-Dizionario

begin

type Dizionario is limited private;
procedure aggiungi (d: in out Dizionario; k: in Integer; v: in Character;
function leggi (d: in Dizionario; k: in Integer) return Character;
procedure cancella (d: in out Dizionario; k: in Integer);
function uguale (d1: in Dizionario; d2: in Dizionario) return Boolean;

private

const max = 100;
type Dizionario = record

chiavi: array {1..max} of Integer;

valori: array {1..max} of Character;

top: 0..max = 0

end record

end package

package body Type-Dizionario

begin

procedure aggiungi (d: in out Dizionario; k: in Integer; v: in Character)

begin
if d.top >= max then raise "Eccesione";

else

begin

for i := 1 to d.top

```

        if  $d.chiori(i) = k$  then return;
        else begin
             $d.top := d.top + 1;$ 
             $d.chiori[d.top] := k;$ 
             $d.valori[d.top] := v;$ 
        end
    end
end
Function leggi ( $d$ : in Dizionario;  $k$ : in Integer) return Character
var  $i$ : Integer;
begin
    for  $i := 1$  to  $d.top$ 
        if  $k = d.chiori[i]$  then return  $d.valori[i]$ 
    raise "Eccezione";
end

```

```

procedure cancella ( $d$ : in out Dizionario;  $k$ : in Integer);
var  $i$ : Integer;
begin
    for  $i := 1$  to  $d.top$ 
        if  $d.chiori(i) = k$  then
            begin
                 $d.chiori(i) := d.chiori(d.top);$ 
                 $d.valori(i) := d.valori(d.top);$ 
                 $d.top := d.top - 1;$ 
            end
        return;
    end
end
end

```

```

Function uguale (d1: in Dizionario; d2: in Dizionario) return Boolean
begin
  if d1.top <> d2.top return false;
  else begin
    var i, j: Integer;
    For i := 1 to d1.top
      if not appartiene(d2, d1.chiavi(i), d1.valori(i)) then
        return false;

    return true;
  end
end

```

```

Function appartiene (d: in Dizionario, k: in Integer, r: in Boolean)
var i: Integer;
begin
  for i := 1 to d.top
    if d.chiavi(i) = k and d.valori(i) = r then return true

  return false;
end

```

...

```

with Type-Dizionario; use TypeDizionario;
var D1, D2: Dizionario;
aggiungi(D1, 1, 'M');
aggiungi(D1, 1, 'S');

```



```

aggiungi(D1, 2, 'S');
aggiungi(D2, 1, 'M');
aggiungi(D2, 2, 'S');
cancella(D2, 2);
if uguale(D1, D2) then put("Sono uguali");
else put("Sono diversi");

```

Definizione di una classe

Generic

Package DizionarioClasse

begin

procedure aggiungiDizionario(k: in Integer; v: in Character)

function leggi(k: in Integer) return Character

procedure cancella(k: in Integer);

end DizionarioClasse

package body DizionarioClasse

begin

const max = 100

var chiavi: array [1... max] of Integer;

var valori: array [1... max] of Character;

var top: range [0... max] := 0;

procedure aggiungiDizionario(k: in Integer; v: in Character)

var i: Integer;

begin

if top = max then raise "Eccezione"


```

else begin
  for  $i := 1$  to  $top$ 
    if  $chiori(i) = k$  then return;
   $top := top + 1$ 
   $chiori(top) := k$ 
   $valori(top) := v$ 
end end

```

```

function leggi( $k$ : in Integer) return Character
var  $i$ : Integer;
begin
  for  $i := 1$  to  $top$ 
    if  $chiori(i) = k$  then return  $valori(i)$ 
  raise "Eccezione"
end

```

```

procedure cancella( $k$ : in Integer)
var  $i$ : Integer;
begin

```

```

  for  $i := 1$  to  $top$ 
    if  $chiori(i) = k$  then begin
       $chiori(i) := chiori(top)$ 
       $valori(i) := valori(top)$ 
       $top := top - 1$ 
    end
  end

```

```

end

```

end Dizionario Classe

with Dizionario Classe;

package D1 is new Dizionario Classe;

with D1; use D1;

package D2 is new Dizionario Classe;

with D2; use D2;

D1.aggiungi(1, 'M');

D1.aggiungi(2, 'S');

D1.cancella(1);

D2.aggiungi(3, 'H');

~~Astrazione generica~~

generic

max: Positive;

type chiere is private;

type valore is private;

package Type Dizionario

begin

type Dizionario is limited private;

procedure aggiungi(d: in out Dizionario; k: in chiere; v: in valore)

function leggi(d: in Dizionario; k: in chiere) return valore

procedure cancella(d: in out Dizionario; k: in Chiere);

function uguale(d1: in Dizionario; d2: in Dizionario) return Boolean
private

type Dizionario = record

chiavi: array[1..max] of Chiave;

valori: array[1..max] of Valore;

top: 0..max = 0

end record

end package

package body Type_Dizionario

begin

procedure aggiungi(d: in out Dizionario; k: in Chiave; v: in Valore)

begin if d.top >= max then raise "Eccezione";

else

begin

for i := 1 to d.top

if d.chiavi[i] = k then return;

else begin

d.top := d.top + 1;
d.chiavi[d.top] := k;
d.valori[d.top] := v;

end

end

function leggi(d: in Dizionario; k: in Chiave) return Valore

var i: Integer;

begin

for i := 1 to d.top

if k = d.chiavi[i] then return d.valori[i]

raise "Eccezione";

end

procedure cancella(d: in out Dizionario; k: in Chiave);

var i: Integer;

begin


```

for i := 1 to d.top
  if d.chiavi(i) = k then
    begin
      d.chiavi(i) := d.chiavi(d.top);
      d.valori(i) := d.valori(d.top);
      d.top := d.top - 1;
      return;
    end
  end
end

```

end

```

Function uguale(d1: in Dizionario; d2: in Dizionario) return Boolean
begin
  if d1.top <> d2.top return false;
  else begin
    var i, j: Integer;
    for i := 1 to d1.top
      if not appartiene(d2, d1.chiavi(i), d1.valori(i)) then
        return false;
      end
    end
    return true;
  end
end

```

```

Function appartiene(d: in Dizionario, k: in Chiave, r: in Valore) return Boolean
var i: Integer;
begin
  for i := 1 to d.top
    if d.chiavi(i) = k and d.valori(i) = r then return true
  end
  return false;
end

```

...

with Type_Dizionario;

package DizionarioIntChar is new TypeDizionario(100, Integer, Character)

with DizionarioIntChar; use DizionarioIntChar;

D1: DizionarioIntChar;

eggyunyi (D1, 1, 'H');